



Universidade Estadual de Campinas – Unicamp
Faculdade de Engenharia Elétrica e de
Computação – FEEC



IA367 - Introdução ao Aprendizado por Reforço

Turma - H

Lista de Exercícios 2

Alunos

João Victor Gomes Carneiro RA 239738

Gabriel Buzzi Sanches RA 256389



Exercício 1

a)

Problema	Usando somente 4 cores, você deve colorir um mapa de forma que duas regiões (ou países) adjacentes não tenham a mesma cor.
Estado inicial	(posição atual, {posição: cor})
Possíveis ações	• Colorir a posição 0 com a cor 1 • Colorir a posição 0 com a cor 2 • Colorir a posição 0 com a cor 3 • Colorir a posição 0 com a cor 4
Modelo de transição	Escolher uma cor válida para um vizinho, mudar a posição atual e o dicionário de cores do estado. O mapa pode ser representado por um grafo/matriz de adjacência.
Função de objetivo	$\text{len}(\{\text{posição: cor}, \dots\}) == \text{len}(\text{mapa})$
Custo de caminho	Custo uniforme.

b)

Problema	Você tem 3 jarros que medem 12 L, 8 L e 3 L, e uma torneira. Você pode: (i) completar os jarros com água até a boca, (ii) transferir o conteúdo de um jarro para outro ou (iii) esvaziá-los descartando seu conteúdo. Deseja-se obter uma quantidade de água de exatamente 1 L.
Estado inicial	(total, [jarro_12, jarro_8, jarro_3]) jarro_i -> quantidade no jarro i
Possíveis ações	Encher jarro, esvaziar jarro, transferir conteúdo de um jarro maior cheio para um menor vazio.
Modelo de transição	Para cada jarro: passar todo o conteúdo para um jarro com capacidade remanescente suficiente; completar totalmente a capacidade remanescente; ou esvaziar completamente o jarro.
Função de objetivo	$\text{sum}([\text{jarro}_i]) == 1$



Custo de caminho Para uma primeira implementação o custo da transição pode ser uniforme. Para uma segunda tentativa, poderia usar a quantidade total atual menos a quantidade alvo (1) como o custo de uma transição, isso deixaria o algoritmo guloso.

Exercício 2

a)

Vetor de estados	(quantidade de missionários na esquerda, quantidade de canibais na esquerda, posição do barco)
Estado inicial	(3, 3, esquerda)
Possíveis ações	Mover o barco de onde ele está para o outro lado com pelo menos uma pessoa
Modelo de transição	• mover barco com 1 canibal se a quantidade de canibais onde o barco está for maior ou igual a 1 • mover o barco com 2 canibais se a quantidade de canibais onde o barco está for maior ou igual a 2 • mover o barco com 1 missionário se a quantidade de missionários onde o barco está for maior ou igual a 1 • mover o barco com 2 missionários se a quantidade de missionários onde o barco está for maior ou igual a 2
Função de objetivo	Alcançar o estado (0, 0, direita)
Custo de caminho	Custo uniforme.

Usando BFS para resolver o problema e salvando cada estado vizinho possível na

forma de dicionário {estado: [vizinhos válidos]} pode-se printar a seguinte árvore de exploração usando uma função que printa o dicionário recursivamente.

Python

```
(3, 3, 0, 0, 'e') <-- Start
├── (3, 2, 0, 1, 'd')
├── (3, 1, 0, 2, 'd')
│   ├── (3, 2, 0, 1, 'e')
│   │   ├── (3, 0, 0, 3, 'd')
│   │   │   ├── (3, 1, 0, 2, 'e')
│   │   │   │   ├── (1, 1, 2, 2, 'd')
│   │   │   │   │   ├── (2, 2, 1, 1, 'e')
│   │   │   │   │   │   ├── (0, 2, 3, 1, 'd')
│   │   │   │   │   │   │   ├── (0, 3, 3, 0, 'e')
│   │   │   │   │   │   │   │   ├── (0, 1, 3, 2, 'd')
│   │   │   │   │   │   │   │   │   ├── (1, 1, 2, 2, 'e')
│   │   │   │   │   │   │   │   │   │   ├── (0, 0, 3, 3, 'd')
│   │   │   │   │   │   │   │   │   │   │   ├── (0, 1, 3, 2, 'e')
│   │   │   │   │   │   │   │   │   │   │   └── (0, 2, 3, 1, 'e')
│   │   │   │   │   │   │   └── (0, 2, 3, 1, 'e')
│   │   │   │   └── (2, 2, 1, 1, 'd')
│   │   └── (2, 2, 1, 1, 'e')
│   └── (2, 2, 1, 1, 'd')
└── (2, 2, 1, 1, 'd')
```

Árvore de exploração do BFS no problema dos missionários e canibais.

b)

Implementamos o algoritmo de busca em largura para solução do problema, código em anexo junto ao relatório. Não seria uma boa checar os estados que se repetem quando se olha os estados como sendo a quantidade de missionários e canibais nas margens, ou transitando no barco. Alguns irão se repetir para garantir a restrição do problema, então essa não é uma boa ideia para colocar como uma restrição ao algoritmo.

A solução e as ações retornadas pelo algoritmo estão descritas a seguir:

Python

Solução encontrada!

Passo 0: Esq(M=3, C=3) Dir(M=0, C=0) Barco: esquerda

Move: 0M e 2C → direita

Passo 1: Esq(M=3, C=1) Dir(M=0, C=2) Barco: direita

Move: 0M e 1C → esquerda

Passo 2: Esq(M=3, C=2) Dir(M=0, C=1) Barco: esquerda
Move: 0M e 2C → direita

Passo 3: Esq(M=3, C=0) Dir(M=0, C=3) Barco: direita
Move: 0M e 1C → esquerda

Passo 4: Esq(M=3, C=1) Dir(M=0, C=2) Barco: esquerda
Move: 2M e 0C → direita

Passo 5: Esq(M=1, C=1) Dir(M=2, C=2) Barco: direita
Move: 1M e 1C → esquerda

Passo 6: Esq(M=2, C=2) Dir(M=1, C=1) Barco: esquerda
Move: 2M e 0C → direita

Passo 7: Esq(M=0, C=2) Dir(M=3, C=1) Barco: direita
Move: 0M e 1C → esquerda

Passo 8: Esq(M=0, C=3) Dir(M=3, C=0) Barco: esquerda
Move: 0M e 2C → direita

Passo 9: Esq(M=0, C=1) Dir(M=3, C=2) Barco: direita
Move: 0M e 1C → esquerda

Passo 10: Esq(M=0, C=2) Dir(M=3, C=1) Barco: esquerda
Move: 0M e 2C → direita

Passo 11: Esq(M=0, C=0) Dir(M=3, C=3) Barco: direita

Exercício 3

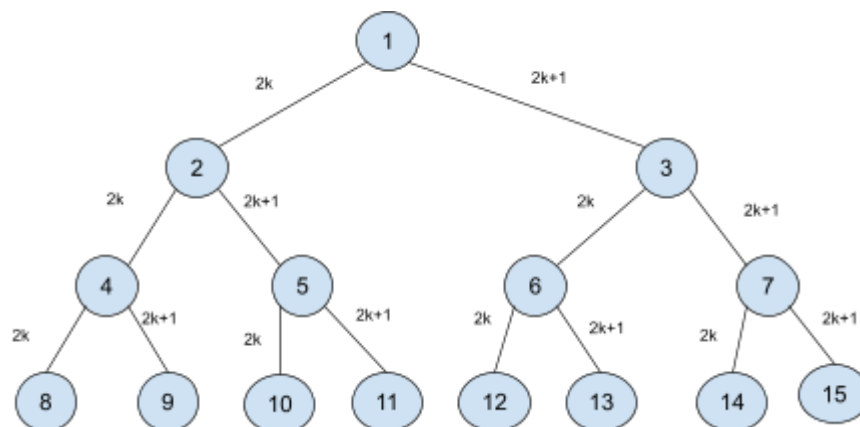
a)

Estado(k)	Sucessor 1(2k)	Sucessor 2(2k+1)
1	2	3
2	4	5
3	6	7
4	8	9
5	10	11
6	12	13
7	14	15

8	16	17
9	18	19
10	20	21
11	22	23
12	24	25
13	26	27
14	28	29
15	30	31

b)

Suponha que o objetivo é chegar a 11. Faça uma lista com os nós visitados para a busca em largura e a busca em profundidade limitada a 3.



Árvore do problema

Busca em largura

O exemplo de código abaixo implementa uma simples busca em largura para este problema e mostra a sequência de nós visitados.

```
Python
from collections import deque
```



```
def bfs(target):
    queue = deque()
    visited = set()
    path = []

    start = 1
    stack.append(start)
    visited.add(start)

    while queue:

        current = queue.popleft()
        path.append(current)

        if current == target:
            return path

        for neighbor in [2*current, 2*current+1]:
            if neighbor not in visited and neighbor <= 15:
                queue.append(neighbor)
                visited.add(neighbor)

print(bfs(11))
# [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
```

Busca em profundidade

O exemplo de código abaixo implementa uma simples busca em profundidade para este problema e mostra a sequência de nós visitados.

Python

```
def dfs(target):
    stack = []
    visited = set()
    path = []

    start = 1
    stack.append(start)
    visited.add(start)

    while stack:
```

```

current = stack.pop()
path.append(current)

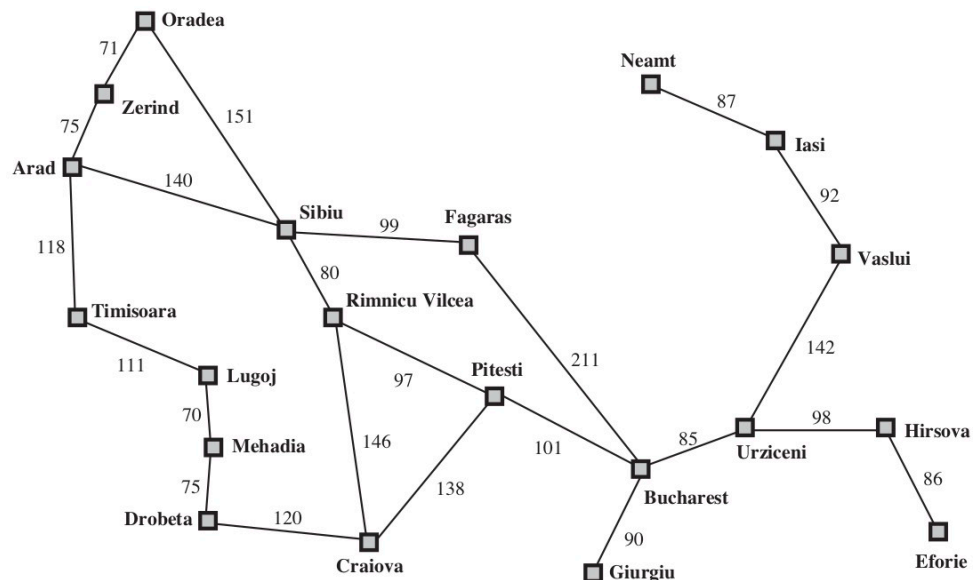
if current == target:
    return path

for neighbor in [2*current, 2*current+1]:
    if neighbor not in visited and neighbor <= 15:
        stack.append(neighbor)
        visited.add(neighbor)

print(bfs(11))
# [1, 3, 7, 15, 14, 6, 13, 12, 2, 5, 11]

```

Exercício 4





Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Mapa e distâncias em linha reta para Bucharest.

Para o algoritmo de busca best-first a função $f(n)$ é definida como:

$$f(n) = g(n) + h(n)$$

Onde para $g(n)$ é o custo do estado inicial até o nó n e $h(n)$ é a distância em linha reta do nó n até o destino final. Para resolver este problema implementamos em python o código em anexo junto com o relatório que retorna a seguinte saída:

Python

```
Iniciando em Lugoj, f=244, g=0, h=244
De Lugoj vá para Mehadia com f=311, g=70, h=241
De Mehadia vá para Drobeta com f=317, g=75, h=242
De Drobeta vá para Craiova com f=280, g=120, h=160
De Craiova vá para Pitesti com f=238, g=138, h=100
De Pitesti vá para Bucharest com f=101, g=101, h=0
```

Exercício 5

Para o problema de 8-queens o algoritmo de busca em largura se mostrou mais eficiente, completando o puzzle com 2056 movimentos enquanto a busca em profundidade precisou de 15720 para encontrar uma solução válida.

A natureza do problema 8-queens faz com que o espaço de busca se torna mais restrito a cada rainha colocada no tabuleiro, então, a grosso modo, o ideal é



avaliar todas as possibilidades de posicionar uma nova rainha dado um estado atual primeiro do que tentar posicionar todas em sequência e se der errado no final voltar atrás nas decisões (comportamento de backtracking do DFS). Por isso o BFS neste caso é mais eficiente, pois é como se antes de tentar posicionar uma nova rainha (avançar de estado) ele já tivesse filtrado as possibilidades que não são interessantes. Em geral para problemas de custo fixo (toda mudança de estado tem o mesmo custo) em que não se tem uma origem e um destino definidos, o BFS é mais eficiente pois ele não “perde tempo” aprofundando em ramos onde a solução não está.

Exercício 6

a)

As heurísticas consideradas foram: 1) soma das distâncias horizontais e verticais de cada peça até sua posição alvo; 2) quantidade de peças fora do lugar; e 3) o máximo entre os valores de 1 e 2.

b)

A heurística 1 leva em consideração o quão longe as peças estão de seus lugares, enquanto que a heurística 2 leva em consideração apenas quantas peças estão fora das posições. Dessa forma a heurística 1 domina a 2, pois seu valor sempre será maior ou igual ao da 2, logo a heurística 3 é simplesmente a mesma coisa que a 1.

Neste caso, a heurística 1 tende a fazer o algoritmo convergir mais rápido uma vez que ela é mais informativa com relação ao estado do problema e o quão perto ele está da solução. Por exemplo, se a peça 8 está trocada com a 1 e o resto das peças estão nas suas posições a heurística 2 teria valor 1 da mesma forma que se a peça 7 estivesse no lugar da 8, entretanto o primeiro caso está muito mais distante da solução que o segundo e essa diferença seria contabilizada pela heurística 1.

Exercício 7

a)

Considerando as posições do tabuleiro existem $9! = 362.880$ opções de jogadas para cada partida. Mas geralmente uma partida pode acabar antes dessas possíveis sequências de permutações. Pesquisando na [internet](#) já é conhecido que existem 255.168 sequências diferentes de movimentos, 31.896 sequências diferentes após eliminar reflexões e rotações das posições finais, 26.830 jogos possíveis se dois

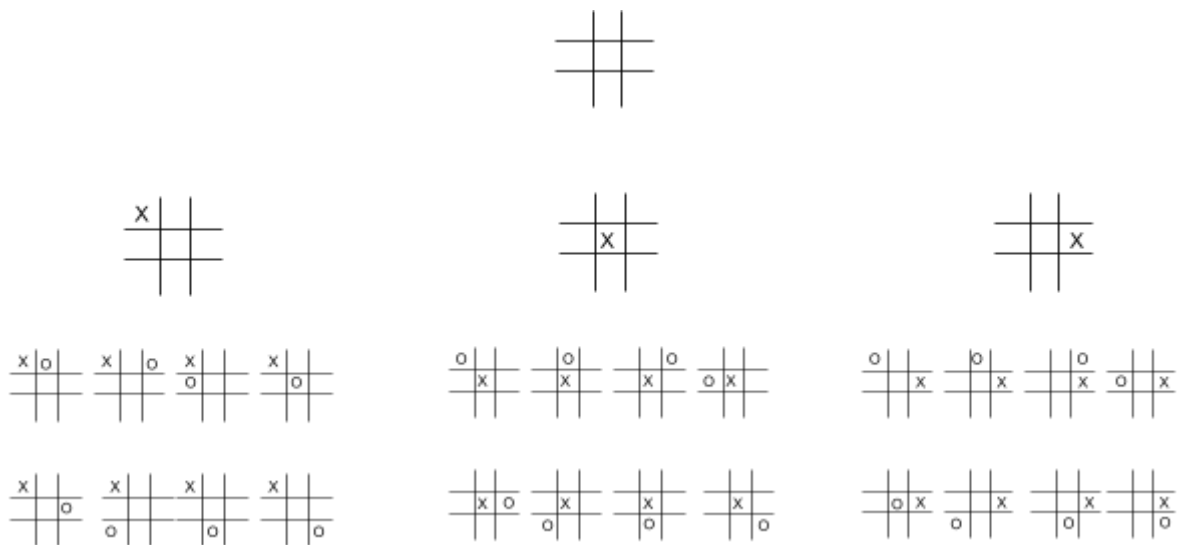


Universidade Estadual de Campinas – Unicamp
Faculdade de Engenharia Elétrica e de
Computação – FEEC



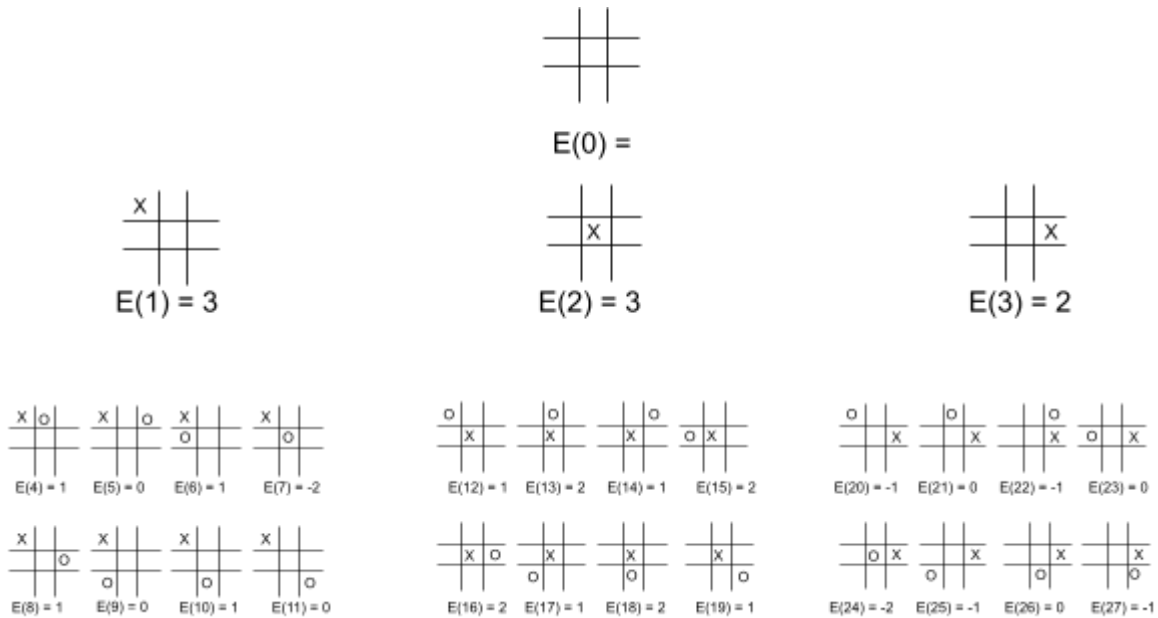
jogos forem iguais quando os jogadores enfrentarem as mesmas escolhas em cada movimento do jogo (ou seja, considerando a simetria em cada etapa do jogo) e 23.129 jogos possíveis sob as mesmas restrições de simetria, se os jogadores desistirem quando o resultado for determinado

b)



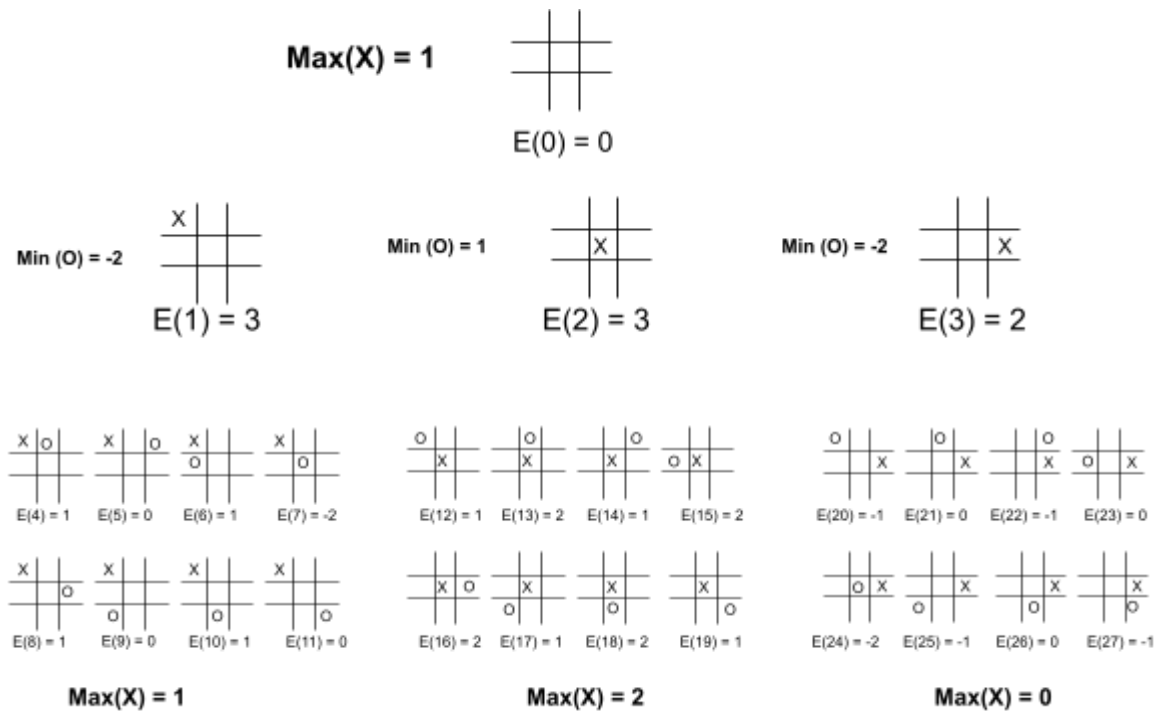
Árvore de jogo completa começando de um jogo vazio indo até a profundidade 2.

c)



Avaliações para todas as posições com profundidade 2.

d)





Usando o algoritmo minimax para escolher a melhor jogada inicial

Portanto para a jogada inicial de **X** o valor que dá a menor minimização para O no nível 1 é o estado E(2), portanto para **X** a melhor jogada inicial é a posição do centro. Considerando para a primeira jogada de **O** no nível 1, a melhor posição de jogada para **O** é qualquer um dos quatro cantos do tabuleiro.

Exercício 8

a)

A primeira saída representa o estado inicial do tabuleiro, com nenhuma jogada, a segunda saída representa o número de vezes que o algoritmo conseguiu encontrar um nó que ainda podia ser expandido e o número de vezes em que uma dessas expansões resultou em empate ou vitória, e a terceira saída representa o estado final depois de executar uma rodada com a árvore “aprendida”, ou seja escolhendo sempre o nó com maior `confidence`.

b)

Executando o código com 100 mil iterações obtém-se valores de `root.plays` e `root.wins` de `10551` e `7597` respectivamente. Isso indica que das 100 mil iterações, apenas 10551 resultaram em uma exploração completa da árvore que levou a um resultado final, e dessas 10551, 7597 foram vitória ou empate.

Ao reduzir o número de iterações do MCTS para 1000 a seguinte saída é obtida:

Python

```
[[ ' ', ' ', ' ', ' '], [ ' ', ' ', ' ', ' '], [ ' ', ' ', ' ', ' ']]  
1000 719  
[[ 'X', 'X', ' ', ' '], [ ' ', 'O', ' ', ' '], [ 'O', ' ', ' ', ' ']]
```

Como se vê, todas as 1000 iterações estão incluídas no valor de `root.plays` isso significa que todas as execuções do MCTS resultaram em uma exploração da árvore que ainda tinha nós disponíveis para serem explorados, ou seja o algoritmo não “terminou” de determinar os melhores valores de `confidence` para todas as possibilidades possíveis de jogadas. Justamente por isso que no final, quando se chama a função `play` o tabuleiro termina incompleto, pois a função `play` apenas



navega pela árvore aprendida escolhendo os nós com maior valor de `confidence` e parando quando um nó não tem vizinhos com `confidence` calculados.

Quando se aumenta o número de iterações para 500 mil obtém-se a seguinte saída:

```
Python
[[ ' ', ' ', ' ' ], [ ' ', ' ', ' ' ], [ ' ', ' ', ' ' ]]
10628 7624
[[ '0', 'X', '0' ], [ '0', 'X', 'X' ], [ 'X', '0', 'X' ]]
```

Este resultado é parecido com o resultado de se executar o algoritmo com 100 mil iterações indicando que com 100 mil iterações o algoritmo já convergiu para uma solução ótima.

Exercício 9

a)

Para $r = 100$ obtém-se a política:

```
Python
# r = 100
Value Iteration:
| 1000.00 || 1000.00 || 1000.00 || 880.23 |
| 1000.00 ||          || 1000.00 || 889.01 |
| 1000.00 || 1000.00 || 1000.00 || 1000.00 |

Policy Iteration:
| RIGHT || RIGHT || LEFT || LEFT |
| UP    ||      || UP    || LEFT |
| UP    || RIGHT || UP    || DOWN |
```

Essa resposta mostra que nenhum dos algoritmos converge para uma solução ótima (que alcança a posição (4, 3), cuja recompensa é 1), pois os valores de recompensa de cada passo são maiores fazendo com que a política deixe o agente preso nestes passos de alta recompensa não atualizando a política nem a utilidade.



b)

Para $r = -3$ obtém-se o segundo resultado

Python

```
# r = -3
```

Value Iteration:

	-5.05		-1.20		2.80		7.63	
	-8.09				0.58		4.99	
	-9.04		-6.25		-2.95		0.36	

Policy Iteration:

	RIGHT		RIGHT		RIGHT		RIGHT	
	UP				RIGHT		UP	
	RIGHT		RIGHT		RIGHT		UP	

Neste caso o algoritmo converge para uma política que prende o agente no estado (4, 3). Neste caso a política ótima faz com que o agente saia de (1, 1) começando tomando a direita e indo por baixo do espaço vazio até (4, 3), o que faz com que ele passe por (4, 2) onde a recompensa é -1, mas neste caso isso faz sentido pois a recompensa dos outros estados é menor que -1.

c)

Para $r = 0$ obtém-se o segundo resultado

Python

```
# r = 0
```

Value Iteration:

	5.88		6.79		7.73		8.92	
	5.16				6.77		6.63	
	4.59		5.15		5.86		5.83	

Policy Iteration:

	RIGHT		RIGHT		RIGHT		UP	
	UP				UP		UP	
	UP		RIGHT		UP		UP	



Agora da mesma forma que no caso de $r=-3$ o algoritmo também converge para uma política que prende o agente no estado (4, 3) mas dessa vez passando pelo caminho de cima de (2, 2), pois passar por (4, 2) onde a recompensa é -1 já não é mais interessante.

d)

Para $r = 3$ obtém-se o segundo resultado

Python

```
# r = 3
```

Value Iteration:

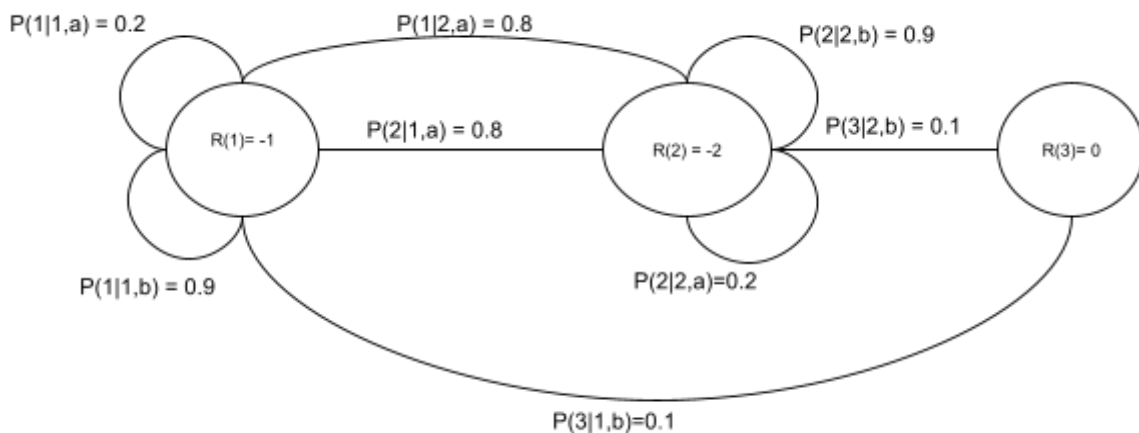
	30.00		30.00		30.00		27.38	
	30.00				30.00		25.76	
	30.00		30.00		30.00		30.00	

Policy Iteration:

	RIGHT		RIGHT		LEFT		LEFT	
	UP				LEFT		LEFT	
	UP		RIGHT		LEFT		DOWN	

Agora, novamente como no caso a) o algoritmo não consegue convergir para uma política que leve o agente para (4, 3), isso porque a recompensa de ficar em outros estados é maior.

Exercício 10



a)

Pode-se afirmar que no estado 1 a ação **b** é mais interessante pois evita o risco de passar pelo estado 2 que tem uma recompensa menor, logo é melhor ficar em 1 até que aconteça a movimentação de menor probabilidade para o estado 3, onde a recompensa é a maior. Já no estado 2 a melhor alternativa sempre será ir para o estado 1 pois ficar no estado 2 é a ação de menor recompensa.

b)

O código implementado se encontra no anexo e a saída é a seguinte:

Python

```
== Policy Iteration ==
```

```
===== Iteração 1 =====
```

```
Política atual: {1: 'b', 2: 'b', 3: None}
```

```
Etapa 1 concluída (Avaliação da política) - Δ total: 1.05e+01
```

```
Valores de estado após avaliação:
```

```
V(1) = -5.263158
```

```
V(2) = -10.526316
```

```
V(3) = 0.000000
```

```
[Etapa 2] Melhoria da política:
```



```
Estado 1: Q(a)={'a': -9.526315786276863, 'b':  
-5.263157893138431} → melhor ação=b  
Estado 2: Q(a)={'a': -7.684210524184575, 'b':  
-10.526315786276863} → melhor ação=a  
Política muda: b → a  
  
===== Iteração 2 =====  
Política atual: {1: 'b', 2: 'a', 3: None}  
Etapa 1 concluída (Avaliação da política) - Δ total: 3.47e+00  
Valores de estado após avaliação:  
V(1) = -5.263158  
V(2) = -7.060334  
V(3) = 0.000000  
  
[Etapa 2] Melhoria da política:  
Estado 1: Q(a)={'a': -7.0308087291460355, 'b':  
-5.263157894653189} → melhor ação=b  
Estado 2: Q(a)={'a': -7.060333761164166, 'b':  
-7.718870346625991} → melhor ação=a  
  
Política estabilizou - encerrando iterações  
  
===== RESULTADO FINAL =====  
Estado 1: melhor ação = b, valor = -5.263158  
Estado 2: melhor ação = a, valor = -7.060334  
Estado 3: melhor ação = None, valor = 0.000000  
Política ótima (PI): {1: 'b', 2: 'a', 3: None}  
Valores (PI): {1: -5.263157894633567, 2: -7.060333761266656, 3:  
0.0}
```

c)

Para o mesmo valor de γ o algoritmo também converge, inclusive para os mesmos valores de $V^\pi(s)$. Porém $\gamma = 1$ o algoritmo não convergiu, pois dessa maneira durante o processo de atualização de $V^\pi(s)$ irá retornar uma série infinita de



penalidades, e como os valores são negativos tende a divergir para $-\infty$. Portanto a política ótima depende do fator de desconto γ .

Exercício 11

a)

Na primeira execução do código a política ótima obtida foi:

←	←	→	↑
↑	X	→	↑
→	→	→	↑

Colocando o agente nesse ambiente para seguir a política aprendida, ele realizou a seguinte trajetória:

Python

Caminho percorrido:

```
[(1, 1), (1, 1), (2, 1), (3, 1), (4, 1), (4, 2)]
```

Ou seja, chegou no objetivo porém com recompensa negativa.

b)

A primeira alteração feita foi no número de iterações, `nTrials=1000`, para permitir que o agente realizasse mais atualizações nos valores de $Q(s,a)$ resultando na política ótima:

→	→	→	↑
↑	X	→	↑
→	→	→	↑

Percorrendo a trajetória



Python

Caminho percorrido:

`[(1, 1), (2, 1), (2, 1), (3, 1), (3, 1), (4, 1), (4, 2)]`

Novamente ele erra duas vezes, batendo na parede nas posições (2,1) e (3,1) respectivamente e chegando no estado final com recompensa negativa.

Diminuindo a taxa de aprendizado, $\alpha = 0.01$, para que a atualização nos valores de $Q(s,a)$ sejam mais sutis a política obtida foi:

←	→	→	↑
→	X	→	↑
↓	↓	↑	←

E novamente caindo no estado negativo com através da trajetória:

Python

Caminho percorrido:

`[(1, 1), (2, 1), (3, 1), (4, 1), (4, 2)]`

Por fim, o último parâmetro testado foi o $nMax = 100$, para permitir que o mesmo estado fosse visitado mais vezes antes de parar de explorar as ações tomadas.

↓	↑	→	↑
↑	X	←	↑
→	→	↓	↓

Python

Caminho percorrido:

`[(1, 1), (2, 1), (3, 1), (4, 1), (4, 2)]`



Novamente não alcançando o estado com recompensa positiva.

O parâmetro que terá maior influência para alcançar o melhor resultado fixando os outros, seria a taxa de aprendizado (α) pois ele diz para o agente o quanto a função de valor ótima das ações deverá ser atualizada durante as iterações. Muito grande o peso das recompensas tende a ter uma contribuição maior porém leva a instabilidade e até não convergência e muito baixo o oposto, pode demorar para convergir ou então empacar em um estado.

Exercício 12

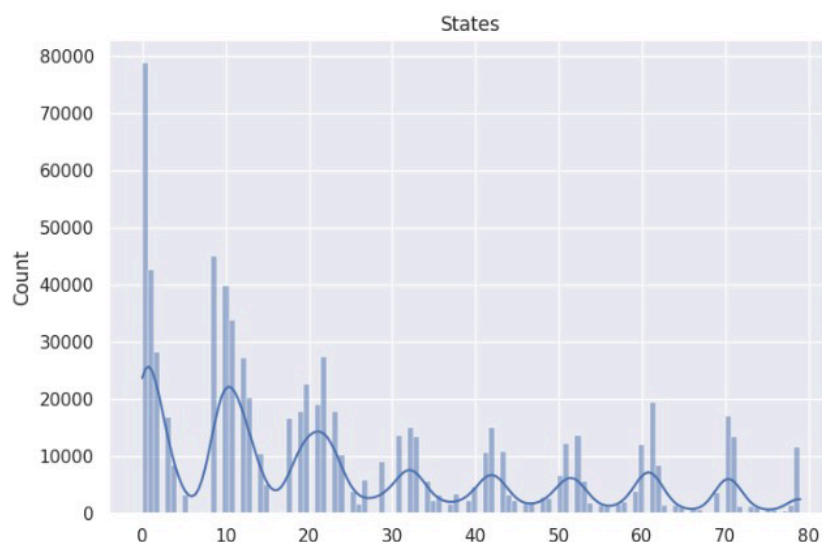
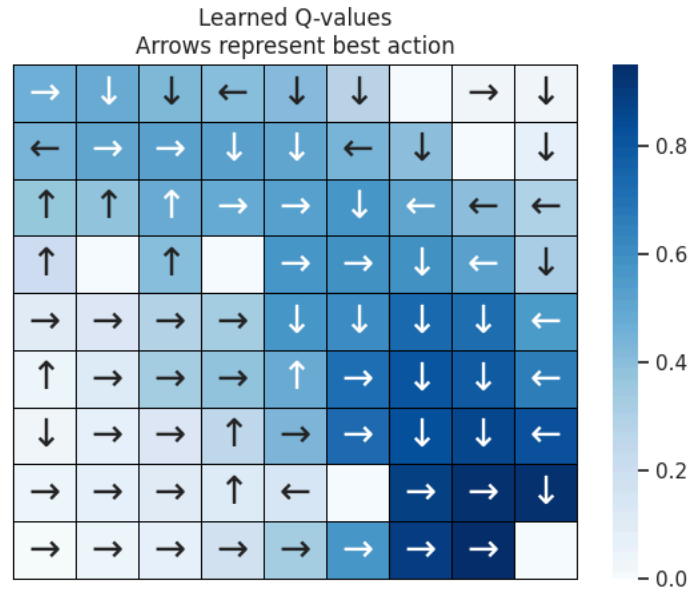


Gráfico States x Count

a)

Os gráficos de contagem de estados mostram quantas vezes um estado foi visitado durante as iterações do algoritmo. Como o algoritmo do Q-Learning vai atualizando a função Q a cada step da simulação (no caso de TD[0]) é natural que os estados com menores valores de Q sejam menos visitados durante toda a execução do algoritmo. Isso é o que resulta neste aspecto de picos e vales no gráfico, os vales representam estados de baixo valor de Q que são pouco visitados durante o aprendizado enquanto os picos representam estados com alto valor de Q que são reforçados durante o aprendizado.



Q-values aprendidos.

b)

O mapa de calor final do grid world mostra algo similar ao mencionado na letra a, mas neste caso mostrando o valor de Q em cada estado no final do aprendizado. Aqui é possível confirmar que os estados menos visitados realmente têm valores de Q menores. Para as células com maiores valores de Q é possível notar também que conforme o agente se aproxima do alvo os valores de Q aumentam confirmando que realmente o Q-learning é uma abordagem de diferença temporal que estima valores de Q diferentes para cada estado levando em consideração a estimativa do valor de Q no estado seguinte.